CLAUDE

Helix Constitution — Universal CLAUDE.md

Field	Value
Revision	5
Created	2026-05-14
Last modified	2026-05-20
Status	active
Status summary	Mirrored Constitution.md §11.4.78 (CodeGraph code-intelligence mandate) into this CLAUDE.md. §11.4.73–§11.4.77 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	§11.4.78 mirror
Fixed summary	§11.4.78 lands in lockstep with the Constitution.md §11.4.78 addition.
Continuation	—

Table of contents

> This is the **base CLAUDE.md** imported by every project that includes the Helix Constitution submodule. Project-level `CLAUDE.md` may extend or tighten any rule by adding an explicit `Override: <section>` block — but MUST NOT weaken them.
>
> Last revision: 2026-05-14

How inheritance works

A consuming project's root `CLAUDE.md` MUST start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/CLAUDE.md
```

```
All rules in `constitution/CLAUDE.md` (and the `constitution/
        Constitution.md`
it references) apply unconditionally. The project-specific rules
        below
extend them.
```

Claude Code supports the `@path/to/file` import syntax natively, so a consuming project can also write `@constitution/CLAUDE.md` at the top of its own CLAUDE.md and Claude Code will resolve it recursively. For agents that do not support `@imports`, the pointer-block pattern above ensures the inheritance is at least readable.

MANDATORY DEVELOPMENT PRINCIPLES

NO BLUFF. Every change ships with positive-evidence validation on the real target environment. A test that passes without exercising the user-visible behaviour is a critical defect.

(Constitution §7.1 + §11.4 — these references point to `constitution/Constitution.md`.)

CRITICAL: All code changes MUST follow these principles WITHOUT EXCEPTION:

1. **Solutions MUST NOT be error-prone.** Every fix must be robust, not introduce new failure modes. If a fix solves problem A but creates problem B, it is NOT acceptable. Test the fix against ALL existing functionality before committing.
2. **No blocking operations inside synchronized / shared-lock regions.** Long computations, network calls, or sleeps inside `synchronized` / `Lock`-held regions cause deadlocks or timeouts in other threads.
3. **Always consider concurrent callers.** Any method called from multiple threads must be safe for rapid consecutive calls.
4. **Test the fix, not just the symptom.** Verify the fix works AND doesn't break anything else.
5. **Anti-bluff is mandatory** (Constitution §7.1) — every runtime test MUST source the project anti-bluff helper, call at least one explicit user-visible action before any PASS, capture state delta, and assert positive evidence.
6. **Real captured evidence for audio/video** (Constitution §7.1 + §11.4.5) — features producing audio output validated via captured audio; features producing video output validated via captured frames + OCR or pixel-diff.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA, integration suites, smoke tests, acceptance suites) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

Canonical authority: `constitution/Constitution.md` §11.4 and its sub-sections §11.4.1 through §11.4.16.

Non-compliance is a release blocker regardless of context.

§11.4.1 — FAIL-bluffs are equally forbidden

A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

§11.4.2 — Recorded-evidence requirement

A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Every PASS for a user-visible feature MUST be cross-checked against captured recording + action timeline.

§11.4.3 — Per-environment-topology test dispatch

Tests that depend on environment topology MUST detect topology at test entry and dispatch the topology-appropriate variant. SKIP-with-reason is the correct fallback when the required topology is absent; PASS-by-default is forbidden.

§11.4.4 — Test-interrupt-on-discovery + retest-from-clean-baseline

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full rebuild → re-deploy on every target → full retest from beginning.

§11.4.5 — Captured-evidence quality analysis

Audio: presence (RMS amplitude), channel count, sample rate + bit depth, glitch census, coexistence-artifact census. Video: presence (non-zero frame count), routing target, frame health (drops / jitter / freeze), obstruction census (OCR for hostile overlays), resolution + codec. Every check is required for every PASS.

§11.4.6 — No-guessing mandate

Forensic anchor — verbatim user mandate (2026-05-08):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Forbidden vocabulary in tests / gates / status reports / closure narratives / commit messages when describing causes: `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`, `guess`, `seemingly`, `apparently`, `perhaps`, `supposedly`, `conjectured`, and synonyms.

Either prove the cause with captured forensic evidence and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

§11.4.7 — Demotion-evidence rule

Demotion from a FAIL classification to a lower-severity classification requires positive evidence captured under the SAME CONDITIONS (same target, same build, same cycle position, same load profile) that originally exposed the defect. "I cannot reproduce in isolation" is a hypothesis, not a finding.

§11.4.8 — Deep-web-research-before-implementation

Before designing a non-trivial fix, perform deep web research: official docs, vendor technical guides, open-source codebases, coding tutorials, issue trackers. Every non-trivial fix's commit message (or accompanying entry) MUST cite at least one external source OR the literal "NO external solution found — original work".

§11.4.9 — Batch-source-fixes-before-rebuild

All source-side fixes that DO NOT require runtime validation to design MUST be landed BEFORE the next artifact rebuild. The anti-pattern of "fix A → rebuild → flash → cycle → fix B → rebuild → ..." serializes operator time onto rebuild latency.

§11.4.10 — Credentials-handling mandate

Forensic anchor — verbatim user mandate (2026-05-12):

> "Credentials or any secret and sensitive data MUST NOT leak!"

Credentials MUST NEVER be tracked in git. `.env` / `.env.*` / `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + README.md exception) git-ignored project-wide. Test scripts MUST NEVER print or log credentials. Per-service file separation limits blast radius. `chmod 600` on credential files, `chmod 700` on parent directory. Rotation on suspected leak.

§11.4.10.A — Pre-store credential leak audit (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> "Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!"

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project's git-ignored configuration, the storing agent MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing: (1) `git ls-files | xargs grep -l <value>` for tree-leaks, (2) `git log -S<value> --all --source --remotes` for history-leaks, (3) surface findings to operator BEFORE storing (operator may rotate, accept-as-compromise, or abort), (4) on finding open a §6/§7 sixth-law-incidents record + redact tracked files in-place to `<redacted-per-§11.4.10>` + record OPERATOR ACTION REQUIRED for rotation per §11.4.10 sub-clause 7, (5) extend pre-push hook credential-pattern grep to catch the escaped class in the same commit. See Constitution §11.4.10.A for the full mandate.

§11.4.11 — File-layout discipline

Project files organised by purpose, not historical accident. Source under canonical project roots. Tests under canonical test directories. Logs and forensic artifacts under operator-controlled directories — never scattered at repo root, never tracked unless they are reference assets.

§11.4.12 — Auto-generated docs sync

Every auto-generated document MUST be regenerated in the same commit as any edit to its source. All output formats (.md + .html + .pdf) MUST stay in sync at all times.

§11.4.13 — Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud monitor, downstream service) provides a network-accessible introspection API that reports what was actually received, the test suite MUST consume that report as captured-evidence. The on-source-side view alone is insufficient.

§11.4.14 — Test playback cleanup

Every test MUST leave the target in a quiescent state. Cleanup mandatory on every exit path (`trap '<cleanup>' EXIT`). The orchestrator MUST run a post-test sanity check and FAIL the just-completed test if it left orphan state.

§11.4.15 — Item-status tracking

Every active item in the project Issues file carries a `**Status:**` line within five lines of its heading. Six-state vocabulary: `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Fixed (→ Fixed.md)`. Status updated as the item progresses. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF).

§11.4.16 — Item-type tracking

Every active item in the project Issues file carries a `**Type:**` line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The Issues_Summary file carries the Type column for every active item. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF). Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION` enforce the mandate.

§11.4.17 — Universal-vs-project classification of new rules (User mandate, 2026-05-14)

Before adding ANY new rule, mandatory constraint, covenant clause, gate, or "MUST"-bearing statement to a project's Constitution / CLAUDE.md / AGENTS.md (or to a submodule's equivalents), the author MUST classify it as **universal** (reusable across any project → goes into this constitution submodule) or **project-specific** (references particular hardware / vendor / package / region → stays in the project / submodule layer). The commit message MUST carry a `Classification:` line stating the choice + one-sentence rationale. Universal rules that leak project-specific assumptions (hardware part numbers, vendor names, geographic regions, internal asset names) MUST be genericised first or downgraded to project-specific. When uncertain, default to project-specific (the narrower scope — lifting to universal later is cheap; the reverse is expensive). Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits new rule commits for the classification statement; paired mutation strips it and asserts gate FAILs.

§11.4.20 — Subagent-driven-by-default mandate (User mandate, 2026-05-14)

When the runtime supports subagent delegation (Claude Code Agent tool, Cursor task-runners, Aider sub-sessions, etc.), the primary agent MUST default to subagent delegation for any task that has multi-step scope (≥3 phases), parallelisable independent

subtasks, long-running diagnostic loops, OR specialised domain workflows (code review, security audit, doc propagation). Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks so quick that subagent overhead exceeds the work. Sub-discipline: **tight scope** (4-6 tasks, not "do everything"), **checkpoint commits** after each major task, **anti-stall protection** explicit in prompts, **anti-bluff verification** of subagent claims via repo state. Parallel subagents MUST partition non-overlapping files; `commit_all.sh --auto-cascade` bundles both via `git add -A`. Gate `CM-SUBAGENT-DELEGATION-AUDIT` (when implemented in consuming project) flags foreground multi-step work as a §11.4.20 violation.

§11.4.18 — Script documentation mandate (User mandate, 2026-05-14)

Every Bash / shell / POSIX-sh script anywhere in a project (`scripts/`, `bin/`, `tests/`, library directories, deployment hooks, CI helpers — depth-N recursive) MUST carry: (1) an in-source documentation block (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) at the top of the file; (2) an external user guide under `docs/scripts/<script-name>.md` covering Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date. When a script is modified, BOTH the in-source block AND the external user guide MUST be updated in the SAME commit. **No documentation ever can be out of sync with its codebase.** Pre-build gate `CM-SCRIPT-DOCS-SYNC` walks every `*.sh` / `*.bash` under script directories, verifies a companion `docs/scripts/<name>.md` exists, AND verifies doc was modified in the same commit (or doc-mtime ≥ script-mtime as a softer floor). Paired mutation strips the doc-sync invariant and asserts gate FAILs.

§11.4.19 — Fixed-document column-alignment mandate (User mandate, 2026-05-14)

Every project that maintains an open-work tracker AND a closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes (Status + Type). For the Fixed archive that means: (1) every `###` / `####` heading in `Fixed.md` (or equivalent) carries a `**Status:**` line and a `**Type:**` line within 8 non-blank lines of its heading; (2) a `Fixed_Summary.md` companion exists with the same column structure as `Issues_Summary.md` (`# | Level | Status | Type | One-line description`); (3) all three file formats (`.md` + `.html` + `.pdf`) for BOTH `Fixed.md` and `Fixed_Summary.md` stay in sync via the same single-shot wrapper that handles Issues + Issues_Summary; (4) closure migration is atomic — when an Issues entry resolves it moves to Fixed.md in the same commit, disappears from Issues_Summary (open-only), and appears in

Fixed_Summary (closed-only). Status values for closed items are drawn from {Fixed (→ Fixed.md) | Fixed — pending device verification | Fixed — RECLASSIFIED}. Type values follow §11.4.16: {Bug | Feature | Task}. Pre-build gate CM-FIXED-COLUMN-ALIGNMENT (5+ invariants) — Fixed_Summary.md exists, table header carries Status+Type columns, mtime (Fixed_Summary ≥ Fixed), generator script present, sync wrapper invokes it, HTML+PDF exports for both Fixed and Fixed_Summary present. Paired mutation strips Status column from Fixed_Summary table header → gate FAILs. Classification: universal (per §11.4.17). No escape hatch.

§11.4.21 — Operator-blocked status + self-resolution exhaustion (User mandate, 2026-05-14)

Operator-blocked is the §11.4.15 Status closed-set's 7th value: {Queued | In progress | Ready for testing | In testing | Reopened | Operator-blocked | Fixed (→ Fixed.md)}. It is a **last-resort classification**, earned only after the agent documents exhaustion of every applicable self-resolution path: (a) CLI / ADB / SSH / API access already available, (b) subagent delegation per §11.4.20, (c) existing repo tooling (scripts / helpers / libraries), (d) captured fallback (synthetic event, asset substitution, mock, §11.4.3 topology SKIP), (e) external research per §11.4.8. Every Operator-blocked item MUST carry an **Operator-Block-Details:** line within 8 non-blank lines of its heading stating: **WHAT** (concrete action), **WHY** (each exhausted alternative enumerated), **UNBLOCK CONDITION** (observable signal), **WHO** (handle / contact / doc pointer). Issues_Summary.md lists Operator-blocked as a sortable Status value. Items MUST be re-evaluated every Nth tag cycle (project- defined, recommended ≥3rd cycle) — operator dependencies change. Fake Operator-blocked (no exhaustion audit) is a §11.4 covenant violation at the planning layer, severity-equivalent to a PASS-bluff. Gates: CM-ITEM-OPERATOR-BLOCKED-DETAILS (every Operator-blocked heading has the details line), CM-OPERATOR-BLOCKED-SELF-RESOLUTION- AUDIT (NEW Operator-blocked commits contain "Attempted: a — ...; b — ...; c — ..." trail). Classification: universal (per §11.4.17). No escape hatch.

§11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes ONLY the status-tracking doc set — Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + any auto-generated audit artifact — so doc-status never drifts behind working-tree reality when the full-repo wrapper is unavailable (in-flight rebase, large submodule churn, partial network). The wrapper MUST: (a) auto-invoke the

project's export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list — NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper's lock; (d) push to every parent-repo remote; (e) exit `3` on nothing-to-commit (informational, not error). The wrapper MUST be invocable standalone OR as a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model. Inherits the project's §9 preflight discipline (refuses to run mid-meta-test). Gate `CM-COMMIT-DOCS-EXISTS` verifies wrapper + guide + flag + doc-set enumeration. Paired mutation strips the doc-set array → gate FAILs. Classification: universal (per §11.4.17). Composes with §11.4.12 (export-sync), §11.4.15 (status tracking), §11.4.18 (script documentation), §12.10 (CONTINUATION maintenance). No escape hatch — doc-status drift is a §11.4 PASS-bluff at the documentation layer.

§11.4.24 — Build-resource stats tracking mandate (User mandate, 2026-05-14)

Every project under this Constitution with a build exceeding 1 minute wall-clock MUST run a host-side resource sampler for every build that captures memory used, CPU%, load average, disk read/write at a fixed interval (recommended 5 s) and computes per-metric **min / max / mean / p95** at stop. Per-build summaries appended to a TSV registry; the registry is the single source of truth — the human-readable Markdown report (and its HTML + PDF exports per §11.4.12) is derived. Top of the report MUST surface **ever-values** (min / max / mean across all tracked builds). Per-build entries sorted most-recent-first; each row carries SUCCESS / FAIL / UNKNOWN + reason for FAIL. Sampler MUST itself stay under 50 MB RSS and 5% CPU (Heisenberg-class observer constraint). Stop hook MUST be called from both success AND failure paths of the build wrapper. The Stats.{md,html,pdf} triple is committed via the project's §11.4.22 lightweight doc-sync wrapper. Gate `CM-BUILD-RESOURCE-STATS-TRACKER` + paired mutation hiding the monitor aside → gate FAILs. Classification: mixed (per §11.4.17) — the discipline universal, the implementation paths project-specific. Composes with §11.4.12 / §11.4.18 / §11.4.22 / §12.6 / §12.7 / §12.9 (the host-safety forensic anchors are the empirical motivation for this telemetry). No escape hatch — build-resource debugging without time-series data is the bluff this anchor forbids.

§11.4.25 — Full-Automation-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product!"

For every consuming project, no feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered **deliverable** until it is covered by automation tests proving six invariants: (1) anti-bluff posture (captured runtime evidence per §7.1 + §11.4); (2) proof of working capability end-to-end on the target topology (per §11.4.3, not in a mock); (3) working implementation matching the documented promise; (4) no open issues / bugs surfaced by the suite (cross-checked against §11.4.15 / §11.4.16 trackers); (5) full documentation (user manual entry + §11.4.18 for scripts) kept in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build

- runtime + paired mutation). Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status), regenerated as part of release-gate sweeps, with gaps tracked per §11.4.15. Classification: universal (§11.4.17). No escape hatch. A project that ships a feature without all six invariants is **not delivering a stable product** — severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. See Constitution §11.4.25 for the full mandate (cross-cutting reach, composition, audit requirements).

§11.4.26 — Constitution-Submodule Update Workflow Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Before ANY modification to `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md`, the agent or operator MUST execute the following pipeline in order:

1. **Fetch + pull first** — inside the constitution submodule worktree run `git fetch` against every remote, then `git pull --`

`ff-only` (or `--rebase` if non-FF-mergeable; never `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization). The submodule MUST be at upstream tip BEFORE any local edit.
2. **Apply the change** — classify per §11.4.17 (only universal additions belong here; project-specific clauses stay in the consuming project's governance). Cite the verbatim user mandate if one originated the change.
3. **Validate before commit** — run `meta_test_inheritance.sh` (or equivalent); verify no merge-conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`); verify Constitution + CLAUDE + AGENTS cross-reference the new clause consistently.
4. **Commit + push to ALL upstreams** — stage only the governance files (NEVER `git add -A` inside the submodule); commit message cites the user mandate + §11.4.17 classification; push to every configured remote. A commit landing on one upstream but not others is a §2.1 violation AND a §11.4.26 violation.
5. **Conflict resolution** — if `pull --ff-only` reports non-fast-forward, merge carefully (preserve union of governance content, no clause silently dropped, re-classify, re-validate). Force-push to "make conflicts go away" is FORBIDDEN (§9.2). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable.
6. **Post-merge validation + verification** — after the push lands, `git submodule update --remote --init` and re-run the consumer project's cascade verifier (per CONST-047) to confirm the new clause reaches every owned submodule. Any cascade gap closed in the same change-window.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer MUST be advanced to the new constitution HEAD in the SAME commit as any cascade work. Out-of-sync pointers are §11.4.26 violations.

Classification: universal (§11.4.17). No escape hatch. A constitution-submodule change violating §11.4.26 is a release blocker for every consuming project, severity-equivalent to a force-push without §9.2 authorization. See Constitution §11.4.26 for the full mandate (operational scope, cross-cutting reach).

§11.4.31 — Submodule-Dependency-Manifest Mandate (User mandate, 2026-05-15)

Every owned-by-us submodule MUST ship a machine-readable dependency manifest at canonical path `helix-deps.yaml` (or .json/.toml) listing its own-org Git SSH dependencies. Schema (per Constitution §11.4.31): `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.recursive: true`, `transitive_handling.conflict_resolution: operator-required`, `language_specific_subtree: bool`.

Tooling: `incorporate-submodule <ssh-url>` adds the submodule at its declared canonical path (CONST-051(C) flat/grouped), reads its helix-deps.yaml, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a §11.4.31 violation.

§11.4.31 is the operational complement of §11.4.28 / CONST-051(C): nested own-org submodule chains are FORBIDDEN, manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Classification: universal (§11.4.17). Composes with §1, §3, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, §11.4.30, CONST-047. See Constitution §11.4.31 for the full mandate.

§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.

Sweep contract (canonical script: `scripts/verify-all-constitution-rules.sh`): re-runs the governance- cascade verifier; for every rule with a programmatic gate (CONST-053 .gitignore audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke), runs the gate against post-pull tree. Failures produce directed FAIL entries → tracker per §11.4.15 with Status: `Reopened`, Type: `Bug`. Closure requires positive-evidence per §11.4 anti-bluff covenant.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook or commit wrapper); operator-explicit manual invocation also available.

Anti-bluff: sweep's own meta-test (paired mutation §1.1) plants a known violation of each enforced gate and asserts sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a §11.4.32 violation.

§11.4.32 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced in the codebase.

Classification: universal (§11.4.17). Composes with every rule that has a programmatic gate. See Constitution §11.4.32 for the full mandate.

§11.4.30 — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!"

Every project module / owned-by-us submodule / service / application MUST ship a proper `.gitignore` covering the forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files (when the generator is committed).
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.nuxt/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.vscode/` (except shared settings), `.history/`.

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be currently tracked. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection: §11.4.30 composes tightly with §11.4.10

- §12.1 (CONST-042) — a `.env` leak is BOTH a §11.4.30 and a §11.4.10 violation, requiring rotation + post-mortem.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it's a build derivative and MUST be ignored. Generators MUST be committed so consumers regenerate on demand.

Classification: universal (§11.4.17). No escape hatch beyond enumerated exceptions. Severity-equivalent to §11.4 PASS-bluff at the repository-hygiene layer. Composes with §1, §2, §9.1, §11.4.10, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, CONST-047. See Constitution §11.4.30 for the full mandate.

§11.4.29 — Lowercase-Snake_Case-Naming Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!"

Every directory, submodule, and file MUST use lowercase snake_case names (ASCII letters / digits / underscores, words separated by _). Existing non-compliant names MUST be renamed as part of the migration window opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a §11.4.29 violation of equal severity to the rename itself.

Exceptions (common-sense, must-not-break-technology). Language- mandated case (Java/Kotlin package paths, Android resource directories, Apple framework dirs, C#/Swift project layouts) is preserved inside the language-root. Submodule root directory follows our convention; language-specific subtree follows its own. Vendor/upstream third-party submodules keep their

upstream names. Build artefacts (`node_modules/`, `__pycache__/`, `.git/`, `target/`, `build/`, `bin/`) keep tool-mandated names. "Does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition. Constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/ `.zshrc`) MUST support BOTH `Upstreams/` and `upstreams/` directory layouts during migration. Lowercase wins when both present. Uppercase fallback retires only by deliberate amendment.

Project-Toolkit Upstreamable synchronisation. Upstreamable / Project-Toolkit machinery MUST be fetched+pulled before any rename batch + MUST itself comply with this rule. Lacking BOTH- directory support is a release blocker.

Test coverage of renames. Each rename batch ships with: (i) regression test verifying every reference now resolves; (ii) full CONST-050(B) test-type matrix run on the post-rename tree; (iii) anti-bluff wire-evidence captured. All three or it's a §11.4.29 violation.

Phased execution. Comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. Phases run in parallel with mainstream work (§11.4.20 subagent delegation).

Classification: universal (§11.4.17). No escape hatch beyond the common-sense exceptions enumerated. Severity-equivalent to §11.4 PASS-bluff at the reference-integrity layer. Composes with §1, §1.1, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, CONST-047. See Constitution §11.4.29 for the full mandate.

§11.4.28 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE

> acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project — directly from project's root project_name/ submodule_name or some more proper structure project_name/submodules/ submodule_name!"

Three cooperating invariants:

(A) Equal-codebase. Every owned-by-us submodule (orgs: `vasic-digital`, `HelixDevelopment`, `red-elf`, `ATMOSphere1234321`, `Bear-Suite`, `BoatOS123456`, `Helix-Flow`, `Helix-Track`, `Server-Factory` — dynamically discoverable via `gh` / `glab` CLIs) is an **equal part** of the consuming project's codebase. Same engineering attention as main: analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, SQL, websites, all materials). A round that improves main while leaving an owned-submodule deficiency unaddressed is a §11.4.28 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. Coverage ledgers (§11.4.25) list every owned submodule as in-scope.

(B) Decoupling / reusability. Owned submodules MUST stay fully decoupled, project-not-aware, reusable, modular, completely testable. NEVER inject project-specific context (hardcoded paths, hostnames, asset names) INTO a submodule. When a submodule needs parent-project info, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Every dependency consumed by an owned submodule MUST be accessible from the parent project's root at:

```
<project_root>/<submodule_name>/
<project_root>/submodules/<submodule_name>/
```

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries pulling in further owned- by-us repos. Add the dependency at the parent's root path; the submodule reaches it via documented import / SDK / runtime resolver. Third-party submodules exempt.

Gates: `CM-OWNED-SUBMODULE-EQUAL-ENGINEERING` (release-gate sweep audits parity), `CM-OWNED-SUBMODULE-DECOUPLING` (pre-commit greps for parent-project context inside the submodule diff), `CM-OWNED-SUBMODULE-LAYOUT` (pre-merge verifies canonical location + no nested own-org submodules + dependency lookup from root). Paired mutations (§1.1) for all three. Classification: universal (§11.4.17). No escape hatch. Composes with §1, §3, §11.4.17, §11.4.20, §11.4.25, §11.4.26, §11.4.27, CONST-047. See Constitution §11.4.28 for the full mandate.

§11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths. Gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` + paired mutation.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/ Challenges submodule fully incorporated), HelixQA (HelixDevelopment/ HelixQA submodule fully incorporated). HelixQA autonomous sessions drive end-to-end execution of every registered test bank with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047):

- Challenges — `git@github.com:vasic-digital/Challenges.git`
- HelixQA — `git@github.com:HelixDevelopment/HelixQA.git`
- Any other functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs the project depends on.

Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7); pointer drift = §11.4.27 violation.

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — this is its strict expansion into per-type-of-test territory). See Constitution §11.4.27 for full mandate.

§11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)

Every project that tracks work items by Type per §11.4.16 MUST close them with the Type-appropriate closure-status word, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, status-counter helpers, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all map to "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate (recommended) `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 (status tracking), §11.4.16 (type tracking), §11.4.19 (Fixed-document column alignment), §11.4.23 (colorisation). Classification: universal (per §11.4.17). No escape hatch.

§11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression,

captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-rediscovered | design-reconsidered }`. Other values are permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`.

A `Reopened` entry without `**Reopened-Details:**` is a §11.4.34 violation. Pre-build gate (recommended) `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 (no-guessing — Reason from closed vocabulary), §11.4.7 (demotion-evidence — reopen IS a demotion from Fixed), §11.4.15 (item-status tracking), §11.4.21 (Operator-blocked discipline — same audit-line pattern). Classification: universal (per §11.4.17). No escape hatch.

§11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)

The constitution submodule's three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the canonical root — also called the parent files. They contain only universal rules per §11.4.17.

The consuming project's repository-root files (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md` or equivalent) are consumer extensions. They open with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading defined in this file's "How inheritance works" section). They contain only project- specific rules per §11.4.17 — rules that reference particular hardware, vendor names, regulatory regions, internal asset names, or project-private conventions.

When in doubt about which file to edit: universal rule → edit constitution submodule's file; project-specific rule → edit consumer's file. Default consumer-side when uncertain (per §11.4.17, narrower scope is cheap to widen).

Terminology: when prose references "the parent CLAUDE.md" or "the root Constitution," the referent is the constitution-submodule file at `constitution/<filename>`, never the consumer's file. When it references "the project CLAUDE.md" or "this project's AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>`. AI agents resolve ambiguous references via this rule.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the commit message.

Pre-build gate (recommended) `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17 (universal-vs-project classification — §11.4.35 defines the file-layer split that §11.4.17 classifies INTO). Reading order: this anchor first, then §11.4.17. Classification: universal (per §11.4.17). No escape hatch.

§11.4.36 — Mandatory install_upstreams on clone/add Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Every clone / add of a Git repository under any consuming project MUST be followed by `install_upstreams` invocation from that repository's root IF its tree contains an `upstreams/` directory (or legacy `Upstreams/` per §11.4.29 transition) populated with `*.sh` recipe files declaring upstream Git SSH URLs.

`install_upstreams` is a host-system utility on operator's `PATH` (exported via `.bashrc`/`.zshrc`), implemented in this constitution submodule (`install_upstreams.sh`). The utility reads recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.

Skipping the invocation when `upstreams/` IS present silently breaks §2.1 (Multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE`

- paired mutation (§1.1).

Automation: `incorporate-submodule` (§11.4.31) and `scripts/init-submodules.sh` patterns auto-invoke `install_upstreams` when applicable. Operator-explicit manual invocation remains supported.

Pre-commit attention: before the first commit in the newly-cloned working tree, verify `git remote -v | grep -c push` reports the expected upstream count.

Classification: universal (§11.4.17). Composes with §2, §2.1, §3, §9.2, §11.4.17, §11.4.20, §11.4.28, §11.4.29, §11.4.30, §11.4.31. See Constitution §11.4.36 for the full mandate.

§11.4.37 — Fetch-before-edit mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!"

The FIRST git-touching action of any session, on any consuming project, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}              # parent
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate (ff-merge / rebase / surface to operator per §11.4.4) BEFORE any local edit, scanner run, or test cycle. Multi-agent / multi-upstream codebases (Claude Code + Cursor + Aider + operator sessions in parallel) routinely lap each other; a 30-second fetch prevents the agent from redoing work a parallel session already finished, from filing a false-confidence "completion" of already-done work, and from doubling the multi-upstream conflict surface (§2.1) with sibling commits of the same change.

The check is non-negotiable even when the operator says "do X immediately" — skipping it on the basis of "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: remote state is not knowable without a fetch. The fetch+log output (even if empty) is the captured evidence.

Scope: consuming project root + every owned submodule recursively (§11.4.28) + the constitution submodule itself (§11.4.26 step 1 made this explicit for constitution-side edits; §11.4.37 generalises to ALL edits) + any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36).

Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit's parent — non-aligned parent = FAIL. Paired mutation (§1.1): synthetic commit whose parent was N commits behind the then-current upstream HEAD → gate FAILs.

Classification: universal (§11.4.17). No escape hatch. Composes with §2.1, §11.4.4, §11.4.6, §11.4.20, §11.4.26, §11.4.32. See Constitution §11.4.37 for the full mandate.

§11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17)

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

A PASS without opening the artifact and verifying the asset chain end-to-end is a §11.4 PASS-bluff. The specific failure mode: source file exists → build packages it → post-build checks pass at the source layer → artifact produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the project's standard QA gate.

Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.38 for the full mandate.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between batch fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — typically 12–48h elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests pass but feature broken" failure mode §11.4 prohibits.

Composes with §11.4.4 (per-fix retest still required at fix granularity) + §11.4.7 (full-suite retest is authoritative baseline for closures) + §11.4.39 (per-feature on-device validation runs as step 3 of the full-suite retest).

Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.40 for the full mandate.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> "make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!"

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per the appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; pre-

viously-passing tests still pass; captured- evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. Both required: CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact — `docs/changelogs/<tag>.md` "Force-push merge-first audit" section captures fetch output, per-remote divergence log, integration strategy, conflict- marker scan, test delta, push output with lease SHA, CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Classification: universal (§11.4.17). No escape hatch. Composes with §9.2, §11.4.4, §11.4.6, §11.4.26, §11.4.32, §11.4.37, §11.4.40, CONST-043, CONST-047. See Constitution §11.4.41 for the full mandate.

§11.4.42 — Iteration-discipline mandate (User mandate, 2026-05-18)

Project work proceeds in priority-ordered iteration cycles. Each cycle has five mandatory steps: (1) select TOP + MIDDLE critical items only (defer LOW until critical batch closed), (2) batch implementation with §11.4.4 four-layer coverage + §11.4.9 batch-source-fixes-before-rebuild, (3) smoke gate (<30 min — batch-touched tests + critical-path regression probe + anti-bluff baseline), (4) ONLY if smoke GREEN and no new operator/user report arrived, run §11.4.40 full retest (12–48 h), (5) release-ready OR loop back to step 1 with new evidence added to queue.

Composes with §11.4.4 (per-fix retest inside step 2), §11.4.7 (closures require same-conditions evidence; step 4 is authoritative baseline), §11.4.9 (source-side batching inside step 2), §11.4.34 (Reopened items attribute source), §11.4.40 (the multi-hour retest IS step 4 — §11.4.42 is the meta-loop conductor).

Anti-bluff coupling: every smoke and full-system PASS MUST carry positive captured evidence per §11.4.2 + §11.4.5. Tests AND HelixQA Challenges bound equally.

No escape hatch — no `--skip-priority-batch`, no `--skip-smoke`, no `--full-suite-only`. Subagents default to the §11.4.42 path.

Classification: universal (§11.4.17). See Constitution §11.4.42 for the full mandate.

§11.4.43 — TDD-Fix-Discipline mandate (User mandate, 2026-05-18)

Every fix MUST follow the 5-step TDD-fix workflow: **RED** (failing test FIRST, real product defect per §11.4.1, captured evidence per §11.4.2) → **LIVE-ADB-PROBE** (try fix on running device via `adb shell` for mutable surfaces — `setprop persist.*`, `settings put`, `pm clear`, `am ...`, boot-script push; INFEASIBLE for kernel / framework / HAL / vendor / init.rc / sepolicy / ro.* / Android.bp — must rebuild; cite `LIVE_PROBE_INFEASIBLE: <reason>` in commit message) → **GREEN** (source patch achieves same effect, batched per §11.4.9, four-layer coverage per §11.4.4) → **VERIFY** (re-run RED test, must PASS under SAME conditions per §11.4.7, captured positive evidence per §11.4.5, 10-iteration reliability per §11.4.42) → **DOCUMENT** (Issues.md → Fixed.md with type-aware closure vocabulary per §11.4.33, CLAUDE.md Applied Fixes row, changelog, guides, HelixQA bank, CONTINUATION.md per §12.10 — all in the SAME commit).

No escape hatch — no `--skip-red-test`, `--no-live-probe`, `--skip-verify` flag. "Test added after the fix" is a §11.4 PASS-bluff: it demonstrates the test agrees with the fix, not that the test catches the bug.

Classification: universal (§11.4.17). Composes with §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.7 / §11.4.9 / §11.4.40 / §11.4.42. See Constitution §11.4.43 for the full mandate.

§11.4.44 — Document revision header mandate (User mandate, 2026-05-18)

Every tracked document in scope (Issues.md, Issues_Summary.md, Fixed.md, Fixed_Summary.md, CONTINUATION.md, docs/guides/**, docs/research/**, docs/scripts/**, docs/changelogs/**, docs/superpowers/plans/**, docs/hardware/**, all other docs/* tracked Markdown) MUST carry a header block directly below the H1 title containing two MANDATORY fields: `**Revision:** N` (monotonic positive integer, never reset, never skipped) and `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` (ISO 8601 UTC). Optional fields (Description, Authority, Maintainer, Scope) encouraged but not gated. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF artifacts are explicitly OUT of scope (revision tracked via VERSION file or auto-derived from source Markdown).

Auto-bump: `scripts/doc_revision_bump.sh <file>` (idempotent), pre-commit hook for automatic staged-doc bumps, `scripts/testing/sync_issues_docs.sh` auto-bumps Issues_Summary / Fixed_Summary after regeneration, `scripts/commit_docs.sh` calls the bump before stage. CONTINUATION.md's existing `Last updated:` line per §12.10 IS the §11.4.44 `Last modified:` line — composed, not duplicated. HTML/PDF exports inherit revision from source Markdown via pandoc pipeline. No escape hatch — no `--skip-revision-bump` flag exists anywhere.

Pre-build gates `CM-DOC-REVISION-HEADER-PRESENT` + `CM-COVENANT-114-44-PROPAGATION` + paired mutations per §1.1. Composes with §12.10 (CONTINUATION.md header reuse), §11.4.12 (Issues_Summary regen), §11.4.22 (commit_docs.sh hook entry), §11.4.23 (HTML colorizer preserves revision), §11.4.18 (companion doc for doc_revision_bump.sh).

Classification: universal (§11.4.17). See Constitution §11.4.44 for the full mandate.

§11.4.45 — Integration-status-doc maintenance mandate (User mandate, 2026-05-18)

Every non-trivial domain integration MUST have a `docs/<domain>/<integration>/Status.md` document that (1) exists when more than one fix/test/gate has landed for the integration, (2) carries the §11.4.44 revision header, (3) is auto-synced (HTML

- PDF) on every related test cycle and every fix touching the integration, (4) is auto-colorized per §11.4.23, (5) has a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh` or a per-integration thin shell, (6) lives under `docs/<domain>/<integration>/`, (7) includes a captured-evidence- driven status table per §11.4.5 (every claim cites the test log / recording / sink-probe report that backs it), (8) uses the closed status vocabulary PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED, (9) lists operator-blocked items at the top so operators find action items in O(1), (10) is referenced from `docs/CONTINUATION.md` §3 (Active work) when any item is non- terminal.

§11.4.45 is the generic form of §12.10 (CONTINUATION.md) applied to every integration domain. Without this generalisation each new integration re-invents the same sync wrapper, revision-header discipline, captured-evidence requirement, and operator-blocked surface.

Pre-build gates `CM-COVENANT-114-45-PROPAGATION` + `CM-AF-INTEGRATION-STATUS-DOCS` + paired mutations (propagation strip / Revision-line delete / sync-staleness / vocabulary violation). Composes with §11.4.5 (captured evidence), §11.4.12 (sync wrapper pattern reused), §11.4.13 (sink-side evidence is a specific in-

stance), §11.4.15 (status vocabulary), §11.4.22 (commit_docs.sh wrapper), §11.4.23 (colorizer), §11.4.44 (revision header), §12.10 (CONTINUATION.md references Status.md paths).

No escape hatch — no `--skip-status-sync`, no `--no-revision-bump-on-status`, no `--allow-stale-html` flag.

Classification: universal (§11.4.17). See Constitution §11.4.45 for the full mandate.

§11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18)

After every device flash, the orchestrator MUST first run a recent-work validation pass (targeted on-device tests for items currently in Issues.md `In progress` / `Ready for testing` / `Reopened`, Fixed.md items closed within last 7 days, CONTINUATION.md §3 active work). Only if that pass is 100% green does the orchestrator proceed to the full post-flash suite.

Helper: `scripts/testing/recent_work_validate.sh --device <serial>` writes `/data/local/tmp/.recent_work_validated` IFF green; the full suite (`test_all_fixes.sh`) refuses to start without it. Marker is in-validated on reboot (stores device boot epoch).

Composes with §11.4.4 (STOP-on-discovery) + §11.4.6 (no-guess-ing) + §11.4.7 (demotion-evidence) + §11.4.40 (full-suite gate) + §11.4.42

- §11.4.43 + §11.4.44 + §12.10. Each recent-item fix MUST have a paired §11.4.43 RED-then-GREEN — a GREEN with no prior RED is a bluff.

Pre-build gates `CM-COVENANT-114-46-PROPAGATION` + `CM-AF-RECENT-WORK-VALIDATION-GATE` + `CM-AF-VALIDATION-ARTIFACT-FILE` + paired mutations.

Classification: universal (§11.4.17). No escape hatch — no `--skip-validation`, `--full-suite-always`, `--ignore-recent-work` flag. See Constitution §11.4.46 for the full mandate.

§11.4.47 — Firebase data review mandate (User mandate, 2026-05-18)

Before every "bigger working round" (pre-build, pre-flash, pre-tag, daily, post-deployment burn-in) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup-maps to existing Issues.md entries via a three-tier algorithm (exact Firebase Issue-ID match → stack-trace- similarity cluster hash → operator merge review), and

drafts new Issue entries for unrecognised findings with full Firebase Console URL refs + §11.4.4(a) systematic-debugging output. Skipping the pass is a §11.4 PASS-bluff — Firebase IS the captured evidence from real end-user devices.

Five mandatory elements: (1) 5-trigger cadence (pre-build / pre-flash / pre-tag blocking, daily / burn-in non-blocking), (2) 3-source query (all three sources), (3) Issues.md output with Firebase metadata (Issue IDs + URL + Cluster Hash / KPI / Funnel), (4) 3-tier dedup, (5) comprehensive systematic-debugging output per Issue.

Pre-build gates `CM-COVENANT-114-47-PROPAGATION` + `CM-AF-FIREBASE-REVIEW-CADENCE` + `CM-AF-FIREBASE-ISSUE-XREF` + 3 paired mutations. Composes with §11.4.4 / §11.4.4(a) / §11.4.6 / §11.4.7 / §11.4.10 / §11.4.12 / §11.4.14 / §11.4.15 / §11.4.16 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.46.

No escape hatch — no `--skip-firebase-review`, `--firebase-review-not-applicable`, `--no-issue-from-firebase` flag. Operator MAY filter with `--severity-min` but MUST execute the pass.

Classification: universal (§11.4.17). See Constitution §11.4.47 for the full mandate.

§11.4.48 — UI-driven video testing mandate (User mandate, 2026-05-18)

Every test that asserts video playback on a secondary display MUST traverse the user-equivalent UI path (launcher icon → app home → content list → tile tap → playback → in-app pause/resume → back button stop). NOT Intent/Broadcast shortcuts (`am start -a VIEW`, `cmd media_session play`). Real `uiautomator dump` element resolution

- `input tap X Y`. Coverage: every video-capable app in PRODUCT_PACKAGES + every stream type (progressive HTTP / HLS / DASH / RTMP / file-local / DRM) + every codec (H.264/265/VP9/AV1/MPEG-2/MP4V video; AC-3/E-AC-3/True-HD/DTS/DTS-HD/MLP/Opus/AAC audio). Secondary display verified via ffprobe-on-captured-mp4 + VOM activeDecoder state. Arvus codec-state cross-check per §11.4.13 + §CG screenshot.

Per §11.4.4 four-layer: pre-build gate `CM-AF-UI-DRIVEN-VIDEO-COVERAGE` + propagation gate `CM-COVENANT-114-48-PROPAGATION` + on-device test framework at `device/rockchip/rk3588/tests/ui_driven/` (Layer 1 helper + Layer 2 per-app drivers + Layer 3 scenarios) + Layer 4 orchestrator `scripts/testing/run_ui_driven_video_suite.sh` + paired meta-test mutations.

Carve-out (User mandate 2026-05-20). The 5 canonical tracker documents — `docs/Issues.md`, `docs/Issues_Summary.md`, `docs/Fixed.md`, `docs/Fixed_Summary.md`, `docs/CONTINUATION.md` — sit at `docs/` root by design. They are architectural constants of the project layout, analogous to AOSP's `Makefile`, `Android.bp`, `OWNERS` files at repo root. Their location is encoded as literal path strings in §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.44 + §11.4.53 propagation gates plus the helper-script constellation that regenerates them. Moving them would require coordinated amendment of those 6 sister anchors plus 5 pre-build gates plus ~20 helper scripts plus 42 consumer files in a single PWU. Per §11.4.66, that scope is operator-blocked until explicitly authorised. Audit-snapshot files (`docs/audit/anti_bluff_audit.md`, `docs/audit/PRE_SONOS_TAG_READINESS.md`, `docs/audit/D1_WIFI_FAIL_CLASSIFICATION.md`, plus any future audit snapshots) DO move under `docs/audit/` per the §11.4.11 general principle.

Canonical authority: constitution submodule `Constitution.md` §11.4.48.

Non-compliance is a release blocker regardless of context.

§11.4.49 — Dual-approach testing mandate (User mandate, 2026-05-18)

Every feature test exercising a user-visible behaviour MUST ship in TWO variants: a UI-driven variant (uiautomator-based, §11.4.48 surfaces A–E) AND an Intent/Broadcast-driven variant (`am start --es` / `am broadcast`-based). Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack — UI catches app-side bugs; Intent catches framework/system-server bugs.

Shared assertion base at `tests/lib/dual_approach_test_base.sh`: `dat_init` / `dat_start_capture` / `dat_assert_codec_state` / `dat_assert_video_frames` / `dat_assert_audio_channels` / `dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_ finding`. Both variants gather identical evidence into mirror directories `qa-results/dual_approach/<F>/<run-ts>/{ui,intent}/` so the orchestrator diffs results and pinpoints which half of the stack contains a bug.

Kinopoisk 5.1 EAC3 is the canonical first implementation. Both variants are RED per §11.4.43 until the §CN decoder pipeline fix lands.

Pre-build gates: `CM-COVENANT-114-49-PROPAGATION` (anchor across parent + 42 consumer files) + `CM-AF-DUAL-APPROACH-COVERAGE` (shared base contract) + `CM-AF-KINOPOISK-5-1-DUAL-COVERAGE` (both variants exist + share the base). Three paired meta-test mutations.

No escape hatch — no `--ui-only` / `--intent-only` / `--skip- dual` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.49.

Non-compliance is a release blocker regardless of context.

§11.4.50 — Deterministic consistency mandate (User mandate, 2026-05-18)

Every test that PASSes MUST have been executed N times (default N=3 normal tests, N=10 cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration. A divergent N-iter run is auto-FAIL — there is no "first PASSed therefore X was a flake" path. §11.4.7's `intermittent` / `transient` / `flake` / `flaky` vocabulary is enforced MECHANICALLY by this mandate, not merely textually.

Coverage: every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit; the threshold ratchets 70 → 85 → 95 → 99 over phases.

Reliability mechanism: `ab_run_n_times <test_name> <N> <fn> [args...]` helper in the project anti-bluff library loops, captures evidence-hash per iter, asserts all N hashes + exit codes identical. NO operator- facing escape converts divergence to PASS.

Pre-build gates: `CM-COVENANT-114-50-PROPAGATION` + `CM-AF-RELIABILITY-CHECK-WIRED` + `CM-AF-FEATURE-COVERAGE-MATRIX`. Three paired meta-test mutations. No escape hatch — no `--allow-flake`, `--first-pass-suffices`, `--skip-n-iter`, `--skip-coverage-audit` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.50.

Non-compliance is a release blocker regardless of context.

§11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18)

Every fix MUST be classified by rebuild-requirement before commit using the project's per-file-class decision matrix. If `LIVE_ADB_TESTABLE` (on-device test scripts, host scripts, atmosphere-*.sh boot scripts, persist.* properties, markdown docs, test fixture assets), the operator MUST first apply the fix to the running device via `adb push` / `setprop` / `pm install -r` / `mount -o remount,rw`, run the §11.4.43 RED test live, capture PASS, THEN commit + rebuild + reflash as belt-and-suspenders re-validation. Commit footer: `LIVE_ADB_VALIDATED: yes`. If `REQUIRES_REBUILD` (kernel, framework Java/AIDL, native C++ in APEX, sepolicy, init.rc, ro.* properties,

XML overlays, codec XML in APEX, Android.bp/.mk), the operator proceeds directly to source-side + rebuild. Commit footer: `REQUIRES_REBUILD: <reason>`. Mixed batches use partial.

§11.4.51 REFINES §11.4.43 step 2 with mechanical enforcement. Helper: `scripts/testing/classify_fix_rebuild_requirement.sh` walks `git diff --name-only`, looks up each file against the matrix, emits per-file classification + recommended commit-message footer. Unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6). Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS`

- `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER`. Three paired meta-test mutations. No escape hatch — no `--skip-classify` / `--assume-rebuild` / `--no-footer-required` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.51.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT a human present to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + JSON result file), headless intent dispatch + state poll (`am start --es`

- `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling), ADB-driven uiautomator (ONLY if `uiautomator dump | grep -c clickable=true` ≥ 1 — near-empty hierarchy proves UI-driven INFEASIBLE and demands fallback to APK/intent), network-side sink probe (Arvus dashboard, Sonos REST, etc. per §11.4.13), HelixQA autonomous QA session (§11.4.27).

Per-feature coverage ledger (§11.4.25) MUST classify each row as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` is a release blocker until promoted, citing a tracked migration work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence per §11.4.5, paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation-coverage — §11.4.52 strictens invariants 1+2), §11.4.27 (no-fakes-beyond-unit + 100% type coverage — operational layer closing the gap), §11.4.39 (per-feature on-device end-user validation — refined to mandate autonomous drivability), §11.4.43 (TDD-fix — autonomous path RED before fix, GREEN after), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach — Intent variant often IS the autonomous path), §11.4.50 (deterministic consistency — autonomous paths scale to N iterations), §11.4.51 (live-ADB-first — instrumentation APK is LIVE_ADB_TESTABLE).

Pre-build gates `CM-COVENANT-114-52-PROPAGATION` (anchor literal across canonical files) + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` (coverage-ledger classification column non-empty + valid value). Paired mutations strip the anchor literal AND inject an `OPERATOR_ATTENDED_ONLY` row without a tracked migration item. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.52.

Non-compliance is a release blocker regardless of context.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

> "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, MUST be executable, MUST emit a markdown table whose header columns include `Status` and `Type` per §11.4.19 column-alignment). Auto- sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH Issues_Summary AND Fixed_Summary in one shot (stages 1a + 1b), then exports HTML + PDF (stage 2), then colorizes per §11.4.23 (stage 3), then re-renders the PDFs from the colorized HTML (stage 3b). MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary parity is the sibling rule; §11.4.12 + §11.4.53 are the canonical pair), §11.4.19 (atomic Issues→Fixed migration triggers Fixed_Summary regen — column-aligned structure), §11.4.23 (visual-cue + grouping colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values literally), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee depends on divergent-summary problem NOT existing).

Pre-build gates `CM-FIXED-SUMMARY-SYNC` (6 invariants: Fixed_Summary exists; HTML + PDF mtime ≥ md mtime; Fixed_Summary mtime ≥ Fixed mtime; generator script exists + executable; sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files + per-consumer propagation). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.54 — ATM-NNN ticket identifier mandate (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every workable item in Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier. ATM- prefix, monotonic, never renumbered, append-only."

Every workable item in `docs/Issues.md` AND `docs/Fixed.md` MUST carry a `[ATM-NNN]` ticket identifier in its heading (form `## §X.Y. [ATM-NNN] <title>`, zero-padded ≥3 digits). Identifiers are allocated by `scripts/testing/assign_atm_ticket_ids.sh` and persisted to an append-only state file `scripts/testing/.atm_ticket_state.json` (jsonl: `atm_id`, `heading_hash`, `type`, `current_location`, `current_status`, `reopens_count`, `created_at`, `last_modified`). Once assigned, an ATM-NNN MUST NEVER be renumbered, reused, decremented, or skipped. `heading_hash` is the binding key — wording reflows preserve the binding via the state-file lookup.

Issues_Summary.md and Fixed_Summary.md MUST carry an `ATM ID` column as the leftmost data column. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`) emit the column so operators / agents can sort + filter on it.

Composes with §11.4.15 (Status), §11.4.16 (Type), §11.4.19 (column-alignment), §11.4.33 (closure vocabulary), §11.4.12 + §11.4.53 (Issues_Summary + Fixed_Summary regen — helper invoked from `sync_issues_docs.sh`), §11.4.55 (per-item Reopens.md path key), §11.4.57 (README.md doc-link cross-reference key).

Pre-build gates `CM-ATM-TICKET-IDS-COMPLETE` (every heading carries `[ATM-NNN]`) + `CM-ATM-TICKET-IDS-UNIQUE` (no duplicates) + `CM-ATM-TICKET-IDS-MONOTONIC` (no gaps) + `CM-COVENANT-114-54-PROPAGATION` (anchor literal across canonical files). Paired mutations strip an `[ATM-NNN]` heading token, dup an ID in the state file, gap the sequence at NNN=2, strip the anchor literal. No escape hatch — no `--skip-atm-assignment`, `--renumber`, `--no-atm-id-required` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.54.

Non-compliance is a release blocker regardless of context.

§11.4.55 — Reopens-history tracking + per-item Reopens.md doc (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a Reopens-count column. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles. Each reopen MUST include By (AI / User), On, Reason, Evidence, and each Fixed-marking with reasoning chain."

Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/<ATM-NNN>/Reopens.md` (+ HTML + PDF). The document MUST contain: (1) §11.4.44 revision header, (2) item identification (ATM ID, Title, Type, Current Status, Current Location, link back to live heading), (3) cycle counters (Total reopens, Total fixed cycles), (4) chronological timeline — one entry per state-change event, each with By (AI/User per §11.4.34), On (ISO date), Event (Opened/Reopened/Fixed/Implemented/Completed), Reason (closed-vocabulary value from §11.4.34 for reopens; captured-evidence summary for closures), Evidence (path or short description), Outcome, (5) reasoning chain for each closure (root-cause analysis, captured-evidence under same conditions per §11.4.7, gate / mutation pair), (6) most-recent state-change pointer.

Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column; cells with count > 0 hyperlink to the per-item Reopens.md. The §11.4.23 colorizer MAY apply a visual cue when reopens > 2.

Composes with §11.4.34 (per-event capture in current heading — §11.4.55 is the per-item history aggregation), §11.4.54 (ATM-NNN provides the stable path), §11.4.44 (revision header), §11.4.45 (Status.md per-integration analogue), §11.4.53 (Fixed_Summary parity — Reopens column symmetric on both summaries).

Pre-build gates `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` + `CM-REOPENS-DOC-REVISION-HEADER` + `CM-REOPENS-COL-IN-SUMMARIES` + `CM-COVENANT-114-55-PROPAGATION`. Paired mutations delete a Reopens.md for a reopens_count=2 item, strip the revision header, remove the column from Issues_Summary, strip the anchor literal. No escape hatch — no `--skip-reopens-doc`, `--collapse-history`, `--reopens-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.55.

Non-compliance is a release blocker regardless of context.

§11.4.56 — Status_Summary parity + two-audience format (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific), page 2 = software engineer summary. Auto-generated after every main Status update."

For every `docs/<domain>/<integration>/Status.md` (§11.4.45) a companion `Status_Summary.md` MUST exist with: (1) §11.4.44 revision header, (2) **Page 1 — For the** — audience- specific heading (audio team for `docs/dolby/*`, video team for `docs/video/*`, etc.) — plain-language summary, What works (1-3 bullets), What's broken or pending (1-3 bullets), Operator / team actions if any. NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names. (3) **Page 2 — For software engineers** — §-letter references, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-references. HTML + PDF exports travel with the markdown.

Generator: `scripts/testing/generate_status_summary.sh <Status.md path>` produces both pages. Invoked from `scripts/testing/sync_integration_status.sh` (§11.4.45 sync wrapper) on every Status.md update.

Composes with §11.4.45 (Status.md per integration — Status_Summary.md COMPLEMENTS, never replaces), §11.4.12 + §11.4.53 (parity discipline), §11.4.44 (revision header), §11.4.23 (colorizer for tracked-item references on page 2), §12.10 (CONTINUATION.md — non-developer stakeholders read Status_Summary; engineers read Status + CONTINUATION + Issues).

Pre-build gates `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` + `CM-STATUS-SUMMARY-TWO-AUDIENCE` + `CM-STATUS-SUMMARY-REVISION-HEADER` + `CM-COVENANT-114-56-PROPAGATION`. Paired mutations delete a Status_Summary.md, remove the Page 1 heading, strip the anchor literal. No escape hatch — no `--skip-status-summary`, `--engineer-only`, `--no-audience-split` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.56.

Non-compliance is a release blocker regardless of context.

§11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Every project's top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (heading MUST contain literal `Tracked-Items`). The section is a markdown table with columns: `Document`, `Last modified` (ISO 8601 UTC from §11.4.44 header), `Revision` (integer from same header), `Markdown` link, `HTML` link, `PDF` link. The section MUST link to: Issues.md + Issues_Summary.md (§11.4.12, §11.4.15, §11.4.16), Fixed.md + Fixed_Summary.md (§11.4.19, §11.4.53), CONTINUATION.md (§12.10), every `docs/**/Status.md` + its `Status_Summary.md` pair (§11.4.45, §11.4.56). Status docs are auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator: `scripts/testing/update_readme_doc_links.sh` extracts each doc's `Revision` + `Last modified`, renders the markdown table, replaces the section between markers (`<!-- doc-link-section:begin -->` / `<!-- doc-link-section:end -->`) in README.md. Invoked from `sync_issues_docs.sh` (Issues / Fixed edits) AND `sync_integration_status.sh` (Status edits).

Composes with §11.4.12 + §11.4.19 + §11.4.53 (Issues / Fixed / summaries — README links all four), §11.4.44 (revision header data source), §11.4.45 + §11.4.56 (Status pairs enumeration), §12.10 (CONTINUATION.md explicit link).

Pre-build gates `CM-README-DOC-LINK-SECTION-PRESENT` (literal `Tracked-Items` + section markers) + `CM-README-DOC-LINK-ROWS-COMPLETE` (every canonical doc appears as a row) + `CM-README-DOC-LINK-FRESHNESS` (`Last modified` matches source within sync-wrapper granularity) + `CM-COVENANT-114-57-PROPAGATION`. Paired mutations strip the markers, remove the CONTINUATION row, backdate a `Last modified` cell, strip the anchor literal. No escape hatch — no `--skip-readme-doc-links`, `--collapse-status-rows`, `--no-freshness-check` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.57.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19T~05:00Z MSK):

> "We MUST DO one comprehensive research and planning in the background in parallel with current mainstream work: our current methodolgy of the development is very slow. … We MUST CREATE adjusted improoved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable

> (or more) item(s) must use parallel agents as much as possible! … Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind! Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components: ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file-scope manifest, §11.4.43 RED test, source patch, pre-build gate per §11.4.4(b) layer 1, post-flash test per §11.4.4(b) layer 3, paired §1.1 meta-test mutation, §11.4.4(b) layer 4 HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52.

5-stage pipeline: (Stage 1 DEVELOP — parallel PWU agents in isolated worktrees) → (Stage 2 MERGE — serial conductor via `commit_all.sh` flock

- §11.4.41 4-step merge-first) → (Stage 3 REBUILD+FLASH — parallel where hardware allows) → (Stage 4 VALIDATE — parallel on D3+D4+meta-test+ coverage) → (Stage 5 SWEEP — parallel HelixQA Challenges + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N — the throughput multiplier.

Synchronization mechanism: 4-layer lock hierarchy (L1 parent flock / L2 per-submodule git ops / L3 contention-path advisory locks for the 10 forbidden cross-PWU paths / L4 per-PWU git worktree). Disjoint-scope PWUs run fully parallel. Conflict detection at Stage 2: `git fetch --all --prune --tags` + scope-overlap check → overlap detected rejects PWU back to Stage 1.

Anti-bluff enforcement at merge time (all four required): C1 §11.4.43 RED-test captured-evidence (proves test catches regression), C2 §1.1 paired meta-test mutation (proves gate is not bluff), C3 §11.4.50 deterministic-consistency (3 or 10 iterations identical), C4 §11.4.5 captured-evidence (audio WAV / video screen recording / UI uiautomator dump / HelixQA `result.json`). Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS all REJECTED.

HelixQA mandatory. Every user-visible PWU MUST add a Challenge entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to its on-device test + scoring PASS only on positive captured evidence per §11.4.5.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE` + `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION` (anchor literal across 42 consumer

files). Paired mutations strip the lock helpers / forge a READY marker without evidence / bypass the merge queue / spawn a 7th concurrent agent / strip the anchor literal — every mutation FAILs its gate.

No escape hatch — no `--skip-merge-queue`, `--allow-bypass`, `--no-anti-bluff-check`, `--unlimited-agents`, `--sequential-phase-chain-mode` flag. Composes with §11.4.4, §11.4.5, §11.4.6, §11.4.9, §11.4.15, §11.4.16, §11.4.19, §11.4.33, §11.4.41, §11.4.42, §11.4.43, §11.4.45, §11.4.49, §11.4.50, §11.4.52, §11.4.54, §11.4.57, §12.6, §12.7, §12.8, §12.10, §9.2.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

Non-compliance is a release blocker regardless of context.

§11.4.59 — README always-sync mandate (User mandate, 2026-05-19)

`README.md` is a §11.4.12-class always-sync document: HTML + PDF exports refresh on every update via `scripts/testing/sync_readme_export.sh` (pandoc + weasyprint); auto-invoked by `sync_issues_docs.sh` so a single doc-sync run refreshes Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README (md + html + pdf). README carries a §11.4.44 revision header and a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation. Pre-build gate `CM-README-EXPORT-SYNC` enforces mtime parity (README.html + README.pdf ≥ README.md). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag. Composes with §11.4.12 + §11.4.18 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.59.

Non-compliance is a release blocker regardless of context.

§11.4.60 — Documentation always-sync composite covenant (User mandate, 2026-05-19)

Eight documentation classes (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md, every Status_Summary.md) MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` that FAILs if ANY instance's

`.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/**` recursively for Status.md fleet. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch — no `--skip-composite-doc-sync`, `--allow-stale-html`, `--summary-not-applicable` flag exists. Composes with §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.23 + §11.4.33 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §11.4.59 + §12.10.

Canonical authority: constitution submodule `Constitution.md` §11.4.60.

Non-compliance is a release blocker regardless of context.

§11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19)

Every workable-item action (open / update / close / reopen / migrate) MUST follow the canonical procedure document at `docs/procedures/issues/<Action>.md`. Closed-set of 5 procedure docs: `Creation.md`, `Updating.md`, `Resolution.md`, `Reopening.md`, `Migration.md`. Each carries §11.4.44 revision header + HTML + PDF exports (refreshed by `scripts/testing/sync_procedure_docs_export.sh`, invoked as the final stage of `sync_issues_docs.sh`). ALL work — new features, behavioural changes, tasks (refactor / doc / infra / gate / audit / cleanup), bug fixes, investigation, documentation — MUST flow through these procedures. No ad-hoc procedure permitted.

Pre-build gate `CM-PROCEDURES-DOCS-PRESENT` checks all 5 procedure docs exist + carry revision header + have current HTML/PDF exports + sync helper is wired into `sync_issues_docs.sh`. Paired mutation: rename one procedure doc → gate FAILs. Propagation gate `CM-COVENANT-114-63-PROPAGATION` enforces this anchor literal in every CLAUDE.md / AGENTS.md.

Canonical authority: constitution submodule `Constitution.md` §11.4.63.

Non-compliance is a release blocker regardless of context.

MANDATORY HOST-SESSION SAFETY (Constitution §12)

Every script, test, helper, and AI agent MUST respect host-session safety. Non-compliance is a release blocker.

Forbidden — directly OR indirectly

1. Suspending the host (`systemctl suspend`, etc.).
2. Hibernating / hybrid-sleeping the host.

3. Logging out the user (`loginctl terminate-session`, `pkill -u <user>`, anything targeting `user@<uid>.service`).
4. Unbounded-memory operations inside `user@<uid>.service` cgroup — any command expected to exceed ~4 GiB RSS MUST be wrapped in a bounded execution scope.
5. Programmatic rfkill toggles, lid-switch handlers, power-button handlers — these cascade into idle-actions.
6. Disabling session managers "to make things faster".

Required safeguards for heavy scripts

1. Source the project's host-safety helper library.
2. Call its pre-flight check and abort if it fails.
3. Wrap any subprocess expected to exceed ~4 GiB RSS in a bounded execution scope.
4. Cap parallelism to fit available RAM.

§12.6 Memory-Budget Ceiling — 60% MAXIMUM

Forensic anchor — verbatim user mandate:

> "First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

Project procedures MUST NOT use more than **60%** of total system RAM. The remaining 40% is reserved for the operator's other workloads. No escape hatch — bypassing this is the bluff §11.4 forbids.

§12.10 Continuation document maintenance

`docs/CONTINUATION.md` MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

MANDATORY ABSOLUTE DATA SAFETY — ZERO RISK (Constitution §9)

Every destructive repository operation (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, object pruning) MUST follow the full §9 safety protocol WITHOUT EXCEPTION:

1. **Backup first, always** — hardlinked mirror of `.git` to a sibling backup directory (`cp -al .git <backup>/repo.git.mirror`) is near-instant and uses zero additional disk.
2. **Record metadata** — refs, tags, submodule pointers, HEAD commit, HEAD tree hash, tree content sha256.
3. **Define expected post-op state**.
4. **Run the destructive operation** — never with `--no-verify`, never with `--force` that bypasses hooks, never auto-force on failure.
5. **Post-op gate** — HEAD tree identical, all tags preserved, all submodule pointers intact, per-entry archive integrity 100%, pre-build gates green. If any check fails → restore immediately.
6. **Force-push authorization** — force-push is NEVER automatic. Each force-push event requires explicit human authorization AND requires the post-op gate to have passed.
7. **Audit trail** — every history rewrite gets a `docs/changelogs/` "Force-push audit" section.

Hardlinked backup is so cheap (zero disk, <2 s) that there is NEVER an excuse to skip it.

MANDATORY COMMIT & PUSH CONSTRAINTS

1. **Use the project's official commit wrapper** for the main repo (e.g. `scripts/commit_all.sh`).
2. **NEVER use `git add`, `git commit`, or `git push` directly** on the main repo unless the project Constitution explicitly carves out a use case.
3. **Multi-upstream push is the norm** — every commit pushed to ALL configured upstream remotes. The constitution submodule ships an `install_upstreams.sh` (and an `Upstreams/` directory) that configures all remotes locally.
4. **NEVER skip hooks** (`--no-verify`, `--no-gpg-sign`) unless the user explicitly authorizes it for the session.

MANDATORY TESTING CONSTRAINTS

Tests MUST be run at every stage WITHOUT EXCEPTION:

1. **Pre-build / pre-merge verification** — BEFORE every build.
2. **Post-build / packaging verification** — AFTER every build.
3. **Post-deploy verification** — AFTER every deploy / flash.

NEVER skip tests. NEVER mark a test as "broken — disable for now" without fixing the underlying issue. NEVER ship a release with unresolved WARNs.

Code conventions

Universal conventions applicable to most projects:

- Use the project's preferred language for new code. Don't mix languages in one module without explicit Constitution permission.
- Static analyzers / linters / type-checkers MUST run clean (zero warnings) before commit.
- Style is set by the project's formatter; do not hand-edit style in PRs.
- Public APIs are documented at the source.

When in doubt

- Read `constitution/Constitution.md` for the canonical text.
- Cross-reference the project's CLAUDE.md / AGENTS.md for project- specific extensions.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff.

---

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA submodules' equivalents. Excludes: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-5.10/**`, `out/**`, `build/**`, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via

`scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be ≥ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION`. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!"

- "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "${BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee …) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules). OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20)

A test asserting audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only. Closed enumeration of acceptable evidence: (1) sink-side codec-state showing non-empty codec matching the expected regex during playback (Arvus / Sonos / Yamaha REST API); (2) PCM `hw_ptr` delta strictly positive across the playback window; (3) ALSA ELD demonstrating negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frames matching expected codec; (5) recording-analyzer matched event per §11.4.2 / §11.4.5 timeline; (6) tinycap RMS amplitude above floor.

Failure modes specifically forbidden: `arvus_probe_present` returning 1 silently dropping the test to SKIP; empty codec-state treated as evidence; `dumpsys media.audio_flinger` policy-side device field used as proof PCM actually opened; config-XML-only PASS without runtime evidence. All four are the §11.4 PASS-bluff pattern materi-

alised on D3 2026-05-20 when the test suite reported audio- routing PASS while the user heard nothing and Arvus Codec-In-Use was empty.

Mandatory protections: (1) sink-side library helpers expose `*_require_reachable` returning exit code 2 (OPERATOR-BLOCKED) when sink unreachable; (2) test harness propagates 2 to summary as OPERATOR-BLOCKED, release tags blocked while OPERATOR-BLOCKED exists; (3) audio-routing tests rewritten to capture ≥1 positive sink/downstream evidence per the enumeration above; (4) anti-stickiness post-stop — test re-probes sink and asserts codec-state transitioned to N.E. / 0-frames-delta. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag.

Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.14 / §11.4.46 / §11.4.49 / §11.4.50 / §11.4.52. Pre-build gates `CM-COVENANT-114-68-PROPAGATION` + `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` with paired meta-test mutations per §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediaco-

dec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.70 — Subagent-driven execution is the default (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution via `superpowers:executing-plans` is permitted ONLY when (a) the task is trivial AND fits in a single sub-300-line edit, OR (b) the operator explicitly requests inline execution at brainstorm-handoff time.

Subagents bring an isolated context window per task (no conductor context bloat), a structurally separated review seam (conductor reviews subagent output, eliminating self-review blind spots), parallel-PWU compatibility (§11.4.58 — subagents ARE the parallel work units), and resumability across operator absence (subagents resume from on-disk plan + spec inputs).

Composes with §11.4.4 (four-layer coverage), §11.4.6 (no-guessing — subagent's captured output IS the evidence), §11.4.42 (iteration discipline), §11.4.43 (TDD-fix), §11.4.50 (deterministic consistency), §11.4.51 (LIVE_ADB_FIRST), §11.4.58 (parallel-development PWU).

No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff. Pre-build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired meta-test mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.70.

Non-compliance is a release blocker regardless of context.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20)

Everyday-push variant of §11.4.41 (force-push merge-first). Before pushing to any upstream for any repository (main repo or submodule), the agent MUST follow the 5-step pre-push cycle: (1) `git fetch --all --prune --tags`, (2) `git pull --no-rebase <remote> <branch>` for each remote whose tip differs from local, (3) investigate the diff vs OUR previous HEAD by reading every foreign commit's body (what changed, why, how it affects OUR project), (4) integrate mandatory changes with full §11.4.4(b) four-layer anti-bluff test coverage producing REAL captured-evidence proofs, (5) THEN push to every configured remote in cascade order.

Composes with §11.4.41 (force-push case) + §11.4.26 / §11.4.32 / §11.4.37 / §11.4.40 / §11.4.42 / §11.4.43 / §11.4.4(b) / §11.4.5 / §11.4.6.

Applies to parent repo + constitution submodule + every owned submodule + every nested submodule + every HelixQA dependency. Audit-trail per push reconstructable from `docs/changelogs/<tag>.md` + per-repo `git log` evidence.

No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. Pre-build gate `CM-COVENANT-114-71-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make sure all fixes for audio are always top priority in main working stream!" The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Audio scope includes §EU HDMI rejection, §EM multichannel HDMI, §ET Arvus integration, §EV/§EW/§EX D3 audio defects, HiFi (Fix #74, #76), AC3 (Fix #106), ES8388 (Fix #103/§104/#105), multichannel LPCM (Fix #112), and every future audio-stack improvement.

Composes with §11.4.42 (iteration-discipline — audio sits at apex of priority order) + §11.4.58 (parallel-development PWU — background research subagents run concurrently but do NOT preempt audio on the main-stream serial dispatch queue).

Operative rule: any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. No escape hatch — no "but this

non-audio task is faster" or "but this research is more interesting" override. Audio-stack regressions are user-perceptible and high-impact (D3 silent post-flash is a release blocker), while research and refactors can wait.

Canonical authority: constitution submodule `Constitution.md` §11.4.72.

Non-compliance is a process violation regardless of context.

§11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!"

Applies only when a project recognises a main specification document (e.g. `docs/specs/**/specification.V<n>.md`). Two-axis versioning: **primary** (V1/V2/V3/…) bumps for major rewrites (old primary archived to `<spec-dir>/archive/`); **secondary** (`Revision`) bumps for additive operator requirements, refinements, polish (matches the §11.4.61 metadata-table `Revision` integer). Every operator-mandated requirement MUST land in the spec as part of the work that implements it. Cross-doc propagation copies MUST reference the active spec file, not a stale archived version. Composes with §11.4.44, §11.4.61, §11.4.59, §11.4.65.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

Non-compliance is a release blocker.

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20)

Direct user mandate (verbatim): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the contributor (human or AI agent) MUST: (1) survey the `vasic-digital` + `HelixDevelopment` orgs on GitHub + GitLab — the

canonical inventory is submodules-catalogue.md (142 repos categorised); (2) reuse an existing Submodule when it covers ≥ 80% of the functionality; (3) extend in-place via upstream PR when 80%+ matches but features are missing — never duplicate the 80% in-project; (4) document the survey result in the relevant tracker row with a `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` line.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

Canonical authority: constitution submodule Constitution.md §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

§11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"

The §11.4 covenant historically relied on agent + operator vigilance. Three forensic incidents in 2026-05-19→20 (two stalled remediation subagents + post-§11.4.66 propagation drift requiring catch-up commit `f93b25a92eb`) demonstrated that late-binding enforcement at `pre_build_verification.sh` time fires hours-to-days AFTER the violator commit has already reached every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline:

1. **Local `pre-commit` git hook** — refuses staged `.md` lacking sibling `.html`+`.pdf` (and other staged-only invariants).
2. **`commit_all.sh` integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before commit (`_constitution_sibling_check` function).
3. **Local `pre-push` git hook** — re-runs siblings + propagation-gate subset on every commit in the push.

4. **`post-commit` auto-repair hook** — detects orphan `.md` in just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **Local-only equivalent** (Phase 39.GF, User mandate 2026-05-20). Remote CI surfaces (GitHub Actions, GitLab pipelines, Jenkins, CircleCI, etc.) are DISABLED — the workflow file is preserved at `.github/workflows/constitution-compliance.yml.disabled-local-only` (NOT `.yml`; GitHub Actions ignores it). Layer 5 enforcement migrated to the LOCAL pre-build-verification + meta-test ritual the operator MUST execute before tagging per §11.4.40. Layers 1-4 remain authoritative; Layer 5 is the operator's local final gate. A future re-enable PWU may re-establish remote CI.

Helper contracts (mandatory): `scripts/install_git_hooks.sh` (idempotent installer wired into `scripts/setup.sh`), `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}`, `_constitution_sibling_check` in `scripts/commit_all.sh`. The `commit-msg` hook enforces a `Bypass-rationale: <reason>` footer when `--no-verify` is detected (touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`); `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Five pre-build gates with paired §1.1 meta-test mutations: `CM-COVENANT-114-75-PROPAGATION` (anchor literal across canonical files), `CM-GIT-HOOKS-INSTALL-SCRIPT` (installer present + executable), `CM-GIT-HOOKS-SOURCE-DIR` (4 hook bodies present + executable), `CM-COMMIT-ALL-SIBLING-CHECK` (`_constitution_sibling_check` in `commit_all.sh`), `CM-CI-WORKFLOW-PRESENT` (CI workflow + ≥3 required jobs).

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI makes the §11.4 promise mechanical), §11.4.41 (this anchor's renumber from §11.4.74 → §11.4.75 was itself driven by §11.4.41 merge-first discipline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive clarification), §11.4.67 (target-shell-parseability — hooks parse under bash AND mksh), §11.4.71 (pre-push fetch + integrate — Layer 3 + 5 honour the merge-first pipeline), §11.4.72 (audio top-priority — hooks hold no lock themselves, so do not race against in-flight audio commits), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch — no `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context.

§11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: `https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

§-slot history note. Originally drafted as §11.4.75, renumbered to §11.4.76 per §11.4.71 fetch-before-push after the concurrent `0a70083` landing of §11.4.75 (Mechanical Enforcement). Same collision-resolution pattern as §11.4.75's own renumber from drafted §11.4.74.

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via `pkg/boot` + `pkg/compose` + `pkg/health` so operators are never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (the **on-demand-infra invariant**); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Composes with §11.4.74 (catalogue-first), §11.4.75 (mechanical enforcement — this clause IS one of the things mechanical enforcement protects), §3 (propagation), §11.4.31 (Submodule-

Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase), §1.1 (paired-mutation gates protect the Submodule's own primitives).

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!"

Forensic anchor. 2026-05-20T15:00Z: ATMOSphere parent's audio Tier 1 `commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` — all untracked but un-gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (download from authoritative source: vendor tarball, SDK installer, npm / pip / cargo / go-mod / container registry, dedicated git submodule, S3/GCS) OR **re-generate** (run from tracked source code via build pipeline, code-gen, asset render, captured-evidence replay, container build, kernel build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) entry in `scripts/setup.sh` (post-clone bootstrap) that runs the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok` recent; (4) README + relevant `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are themselves a §11.4 PASS-bluff variant — codebase appears complete to casual eye but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling + verifies `script-path` exists + parses YAML + checks bootstrap reference. Paired §1.1 mutation strips one required YAML key → gate FAILs.

Composes with §11.4.6 (no-guessing — mechanism MUST be verified working on sandbox clone, not assumed), §11.4.65 (universal Markdown export — generated HTML/PDF siblings are a `regenerate` instance), §11.4.66 (interactive clarification — ASK operator if mechanism unknown), §11.4.71 (pre-push fetch + integrate — re-validate vendor integrity before push), §11.4.74 (catalogue-first — extend a reusable downloader Submodule rather than re-implement), §11.4.75 (Mechanical Enforcement — pre-commit + CI replay layers refuse bare additions), §11.4.76 (Containers-submodule — container images regenerated via `vasic-digital/containers`), §9 / §9.2 (zero-risk data safety — pre-test mechanism in sandbox with hardlinked-backup), §3 (propagation — consuming submodules inherit §11.4.77).

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm `@colbymchenry/codegraph`) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud, no external API). (1) Install globally via npm — no `sudo` (the npm prefix MUST be user-writable). (2) `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude other-owned submodules AND — non-negotiably — every credential/secret path per §11.4.10 (a secret reaching the index is a §11.4.10 violation). (3) Wire the `codegraph serve --mcp` server into every CLI agent the developers use — project-scoped + committed where supported (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`), host-local otherwise (Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH` (no hardcoded host path). (4) Cover the integration with an anti-bluff verification suite whose per-agent end-to-end layer uses an **unforgeable challenge** — a fact obtainable only by calling a CodeGraph MCP tool (e.g. the index node count via `codegraph_status`) so an agent answering from its own file-reading tools cannot produce a false PASS; an agent that genuinely cannot be driven end-to-end (missing credentials / quota / environment incompatibility) is a documented SKIP gap per §11.4.3, never a faked PASS. (5) Document

everything in `docs/CODEGRAPH.md`, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not added as a git submodule, and it adds no Git remote.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4 (anti-bluff — CI green is necessary, never sufficient), §1.1. Planned gate `CM-CODEGRAPH-WIRED` + paired §1.1 mutation (strip a secret-exclusion from `config.json` → gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach of this mandate.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

Refines §11.4.78 step 2's exclude-list with a per-submodule-ownership split: (a) **Own-org submodules** — full write access via the project's CLIs (canonical orgs: `vasic-digital` on GitHub/GitLab/GitFlic/GitVerse + `HelixDevelopment` on GitHub) — MUST be **INCLUDED** in the index. (b) **Third-party submodules** (the §11.4.74 `no-match → vendor` path, e.g. `gopkg.in/telebot.v3`) — MUST be **EXCLUDED**. Operational steps: (1) `git submodule update --remote --merge` to pull latest from every submodule before re-indexing — respect load-bearing pins on third-party submodules (e.g. roll back if `--remote` advances `gopkg.in/telebot.v3` past the v3 import-path pin). (2) Adjust `.codegraph/config.json` exclude list to keep own-org paths in scope. (3) Re-index via `scripts/codegraph_setup.sh`. (4) Verify via `scripts/codegraph_validate.sh` with at least one probe that resolves a symbol living ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore.

Composes with §11.4.74 (catalogue ownership classifier), §11.4.78 (CodeGraph parent mandate), §11.4.10 (credentials still excluded regardless), §1.1 (paired mutation), §107 (an index that lies about reachable symbols is a §107 PASS-bluff against AI agents).

Canonical authority: constitution submodule `Constitution.md` §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded WITHOUT an audit trail in `.codegraph/config.json` comments) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! [...] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule since they MUST BE available to all projects which do respect and follow our constitution rules and mandatory constraints! Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents (another area / context to regularly update and sync) and regularly export them like all other Status docs into the PDF and HTML!"

Three deliverables (all in this constitution submodule): (1) `scripts/codegraph_update.sh` — npm-installs latest `@colbymchenry/codegraph` after npm-registry version check; appends old/new version to `docs/codegraph/Status.md`; §107 anti-bluff: verifies `codegraph --version` reflects the new version after install (npm exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update, runs `codegraph status` → `codegraph sync .` → `codegraph status` → project's `scripts/codegraph_validate.sh` in the consuming project; appends every step's output to BOTH the project's and the constitution's `docs/codegraph/Status.md`. (3) `docs/codegraph/Status.md` + `Status_Summary.md` append-only ledgers tracking all CodeGraph-related events; exported to `.html` + `.pdf` siblings per §11.4.65.

Operational cadence: weekly floor (per §11.4.45 status-digest cadence). Scripts are inherited by reference (per §3 submodule inheritance) — consuming projects invoke them at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy.

Composes with §11.4.78 (CodeGraph parent), §11.4.79 (own-org submodule inclusion), §11.4.10 (credentials excluded), §11.4.45 (status-digest cadence), §11.4.53 (Fixed_Summary backfill), §11.4.65 (multi-format export), §107 (observed-version evidence per update; observed PASS/FAIL per sync), §1.1 (paired mutation: downgrade installed version → script detects drift → restore).

Canonical authority: constitution submodule `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (consuming project has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation that depends on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED.** Linux cgroup/systemd//`proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit`/`ulimit`, macOS `launchd`, BSD `rctl`, Windows Job Object) chosen via runtime dispatch — same operator-path invocation, OS-correct mechanism. **(B) Per-OS tests REQUIRED.** Every gate test that exercises platform-dependent behaviour MUST have `case "$(uname -s)" in` branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch. SKIP-with-reason is acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED.** Where a Linux primitive has NO macOS/other-OS equivalent due to a documented kernel limitation (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + link to honest-gap doc, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. `RLIMIT_CPU+SIGXCPU` as the macOS proxy for "process is bounded"). The adjacent test MUST itself be anti-bluff with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical, not exhaustive): `systemd-run --user --scope` ↔ POSIX `ulimit -t -u` / launchd; cgroup `MemoryMax` ↔ XNU gap (use `RLIMIT_CPU` adjacent) / `rctl` / Job Object; cgroup `TasksMax` ↔ `RLIMIT_NPROC`; `/proc/<pid>/oom_score_adj` ↔ no equivalent on Darwin/BSD (no kernel OOM-killer).

Composes with §11.4.1 (FAIL-bluffs forbidden — Linux-PASS / Darwin-SKIP without honest reason is a §11.4.81 violation), §11.4.2 (recorded evidence per branch), §11.4.3 (§11.4.81 strictens §11.4.3: topology SKIP only when kernel CANNOT, not when "we didn't implement yet"), §11.4.4 (test-interrupt triggers when discovering a Linux-only test without per-OS equivalent), §11.4.5 (captured evidence per platform), §11.4.6 (no guessing about platform capability — prove via reproducer), §11.4.20 / §11.4.70 (per-OS branches are parallel-subagent dispatchable), §11.4.27 (100% test-type coverage applies per-platform), §11.4.69 (sink-side evidence per platform), §107 (multi-platform projects must guarantee end-user usability on every platform). Pre-build gate `CM-CROSS-`

`PLATFORM-PARITY` (when implemented): scans every gate test for `case "$(uname -s)"` blocks, asserts a non-SKIP branch (or honest-gap citation per (C)) exists for each platform in the supported-platforms manifest. Paired §1.1 mutation: strip a Darwin branch → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects. No escape hatch.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Forensic anchor — direct user mandate (verbatim, 2026-05-22): "How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!"

Iteration cycle time is a first-order quality enabler. Slow cycles bound the project's defect-discovery rate; fast cycles multiply it. The 2026-05-22 forensic session witnessed ~3 hours of operator wait time on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following speedup disciplines AS MANDATORY (each independently enforceable):

- **(A) Phase 1 forensic before any speculative source patch** — before any non-trivial source patch, complete `superpowers:systematic-debugging` Phase 1 to identify FACT-grade root cause. Speculative patches without Phase 1 evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any rebuild** — strengthens §11.4.51 to a release-blocker mandate. Skipping live-probe for a LIVE_ADB_TESTABLE change is a §11.4.82 violation.
- **(C) Pre-flight before launching rebuild orchestrators** — 30-second pre-flight verifies device reachability, sink reachability, host memory/disk, no stale locks, no orphan processes. A rebuild against a broken precondition wastes 45 min.
- **(D) Persistent build caches outside containers** — `ccache` + Soong / sccache / Gradle daemon state bind-mounted to host, NOT in ephemeral container layer. Cold rebuild ~40 min → warm rebuild 5-15 min.
- **(E) Module-only rebuild for loadable-module-only changes** — `make -C kernel M=<driver-path> modules` for `CONFIG_*=m` drivers (~2-5 min). Full rebuild required only for `=y` built-ins.

- **(F) Parallel multi-device testing** — every owned validation device runs the autonomous cycle concurrently with separate `qa-results/<TS>/<device-tag>/` outputs.
- **(G) Subagent scope discipline + worktree isolation** — subagents ≤30 min budget, single-responsibility, intermediate output emitted as they progress, `isolation: "worktree"` by default.
- **(H) Lock-file + stale-process hygiene** — detect and clean `.git/index.lock`, `.lock` files, orphan `git`/build processes on session start. Disable auto git-gc (`gc.auto 0`) in concurrent multi-agent repos.
- **(I) Cycle telemetry per §11.4.24** — every iteration logs commit hash, per-phase wall-clock, speedup-discipline flag set, outcome. Aggregated weekly to surface which disciplines deliver the biggest empirical return.

Composes with §11.4.4, §11.4.6, §11.4.9, §11.4.20 / §11.4.70, §11.4.24, §11.4.42, §11.4.43, §11.4.50, §11.4.51, §11.4.52, §11.4.58, §12.7, §107. Pre-build gate `CM-ITERATION-SPEEDUP-DISCIPLINE` (when implemented): audits the most recent N cycles for telemetry citing which of (A)-(I) applied. Paired §1.1 mutation: strip the speedup-flag column → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.82.

Non-compliance is a release blocker. No escape hatch — no `--skip-phase1-forensic`, `--no-pre-flight`, `--rebuild-everything-always`, `--unlimited-subagent-scope`, `--ignore-locks`, `--no-telemetry` flag exists.

§11.4.83 — docs/qa/ end-user evidence mandate (User mandate, 2026-05-22)

Forensic anchor — verbatim user mandate (2026-05-22):

> "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. A feature with no QA transcript is itself a §11.4 / §107 PASS-bluff: it claims to

work but has no auditable runtime evidence that an end user actually exercised the feature through the same interface they will use in production.

Operative rule. (1) Every consuming project MUST maintain a `docs/qa/` tree. Each new feature lands under `docs/qa/<run-id>/` where `<run-id>` is monotonic + greppable (timestamp, HRD-NNN, ATM-NNN, other workable-item ID per §11.4.54). (2) Transcripts MUST be full bidirectional — every prompt/command sent + every response received. One-sided is not a transcript. (3) Attached materials MUST be committed in-repo (no external-only links — that is §11.4.13 sink-side violation). (4) Bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact. (5) CI / release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.83.

Non-compliance is a release blocker. No `--qa-evidence-optional` escape hatch.

§11.4.84 — Working-tree quiescence rule for subagent commits (User mandate, 2026-05-22)

Short tag: `working-tree quiescence`.

Forensic anchor — verbatim user mandate (2026-05-22):

> "no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add`, the committing agent MUST grep its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, `// MUTATION` / `# MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area. Any unexplained file → ABORT.

Lesson (forensic case study). A consuming project's logo-fix subagent (Herald commit `72e81ab`, 2026-05-21) ran in a checkout where a paired §1.1 mutation gate had temporarily introduced an

`// always pass` shortcut into a JWT verify path. The subagent's `git add` swept the mutation residue into the same commit as the unrelated logo fix, and the resulting commit was pushed to all four mirrors before any other agent caught it. The fix (Herald `d5bd360`, "SECURITY FIX: restore commons_auth/middleware.go JWT verify") landed within the hour, but the window during which production-equivalent binaries shipped with a bypassed JWT verify is a real security-defect window. The lesson is now constitutional.

Operative rule. (1) Pre-`git add` MUST grep for mutation markers + cross-check `git status --porcelain` against the subagent's declared scope; unaccounted entries → ABORT. (2) Any active mutation gate MUST be serialised — mutate → assert FAIL → restore → assert PASS — and the working tree MUST be verifiably clean BEFORE any unrelated commit. (3) Concurrent subagents in the SAME checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`); the cleaner solution is `git worktree add` per subagent (composes with §11.4.20/§11.4.70). (4) Post-commit `mutation-residue-scanner` MUST run before push; any commit containing a mutation marker → push BLOCKED.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: constitution submodule `Constitution.md` §11.4.84.

Non-compliance is a release blocker. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Short tag: `stress-chaos-mandate`.

Forensic anchor — verbatim user mandate (2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock, per-iteration latency p50/p95/p99 recorded) + concurrent contention (N ≥ 10 parallel invocations, no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input — every boundary produces a categorised result).

Chaos (closed-set, mechanically auditable, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call, categorised recovery) + network-fault injection (drop/delay/reorder, `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test, detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade gracefully, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault — recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason` per §11.4.69. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT`; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (test files exist + executable + sh -n + bash -n parseable; library exists; fix's pre-build gate cites the stress+chaos test path) + paired meta-test mutation per §1.1 (strip chaos-injection or evidence-capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51, dispatched against a real device with captured evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden — set-u crashes from chaos step are bluffs), §11.4.5 (captured-evidence content quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations produce identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists.

§11.4.86 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2026-05-25)

Forensic anchor — verbatim user mandate (2026-05-25):

> "Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"

Some Status docs (§11.4.45) are backed by a **tracked roster** (installed apps/components) or a **tracked asset corpus** (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (player app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync **out of the box**, mechanically.

Mechanism (all must hold): (1) **drift-proof fingerprint** — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) **sync helper** that regenerates the fingerprint + re-exports HTML+PDF (via the §11.4.65 exporter), wired so sync is automatic; (3) **pre-build gate** that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) **paired §1.1 mutation** that corrupts the fingerprint and asserts the gate FAILs.

Composes with §11.4.12 / §11.4.45 (roster/corpus specialisation) / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Classification: universal (§11.4.17) — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Canonical authority: constitution submodule `Constitution.md` §11.4.86.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When the operator instructs an AI agent to "continue in endless loop fully autonomously" (or any semantically-equivalent phrasing), the agent MUST treat this as a HARD-CONTRACT covenant: (A) continue working until `docs/Issues.md` Status-column has zero non-terminal entries AND `docs/CONTINUATION.md` §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, "waiting for results" is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs — "physical proofs"); (D) the §11.4 anti-bluff covenant family (§11.4.1/§11.4.2/§11.4.6/§11.4.7/§11.4.27/§11.4.50/§11.4.52/§11.4.68/§11.4.69/§11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally per the historical forensic anchor "we had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used"; (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.87.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag exists.